18 Bluetooth Controllers Walk Into a Bar

Observability & Runtime Configuration with CNCF Tools

Simon Schrottner
Dynatrace

Manuel Timelthaler
Tractive

KubeCon + CloudNativeCon Europe 2026

What is JoustMania?

A motion-controlled party game for up to 18+ PlayStation Move controllers

Keep your controller still, jostle everyone else's.
No screens, just glowing controllers and chaos.

Photos by Brent Knepper, Sara Bobo, and Die Gute Fabrik;
showing original J.S. Joust game

How do we know what's going on?

What if we added observability tools?

What if we added feature flags too?

That's the experiment
CNCF tools on a real-time party game

- What breaks at real-time speeds?
- What actually works out of the box?
- What needs tuning, and how much?

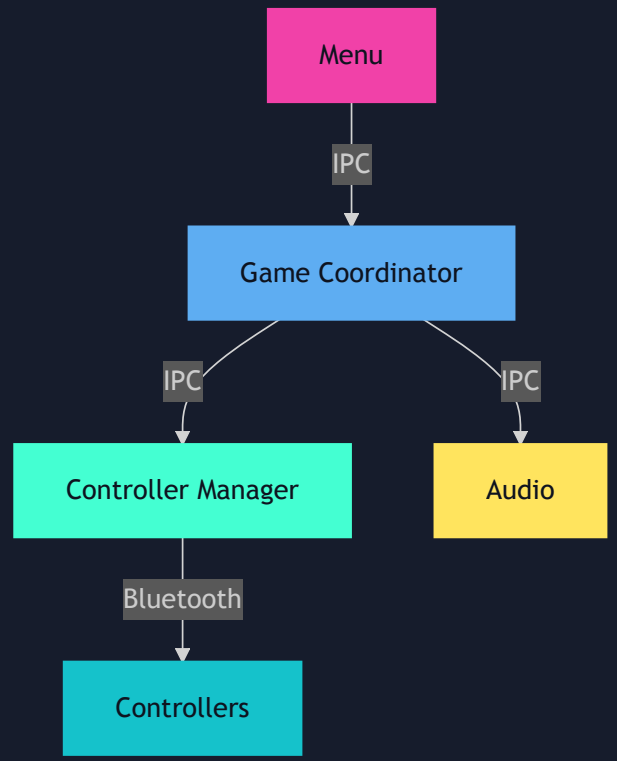
The Journey: 6 Learnings

6 key learnings from instrumentation to subsecond metrics

Learning 1: The Original Architecture

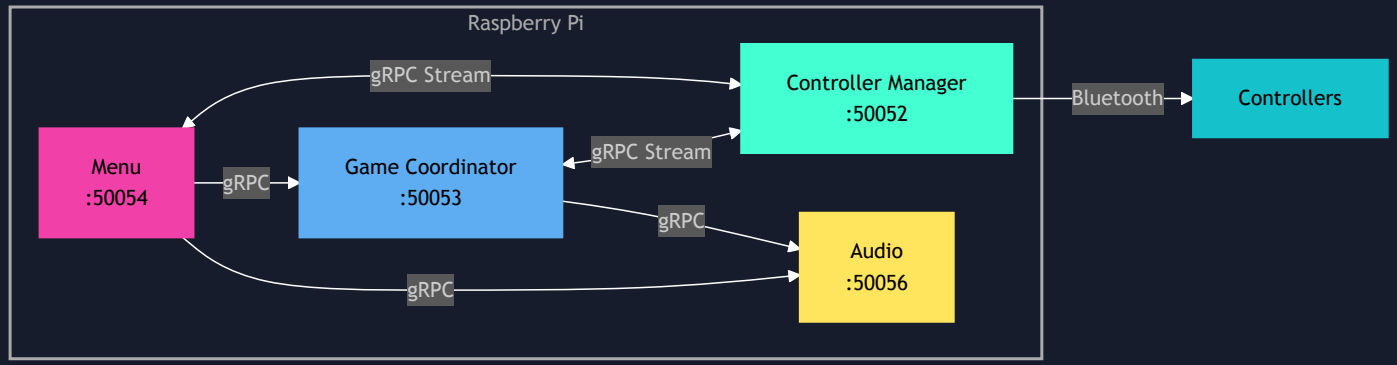
- Process-based (multiprocessing)
- 60 Hz game loop (16ms frames)
- IPC via queues/shared memory
- 4 Python processes

Problem: IPC (pipes/queues) needs manual instrumentation



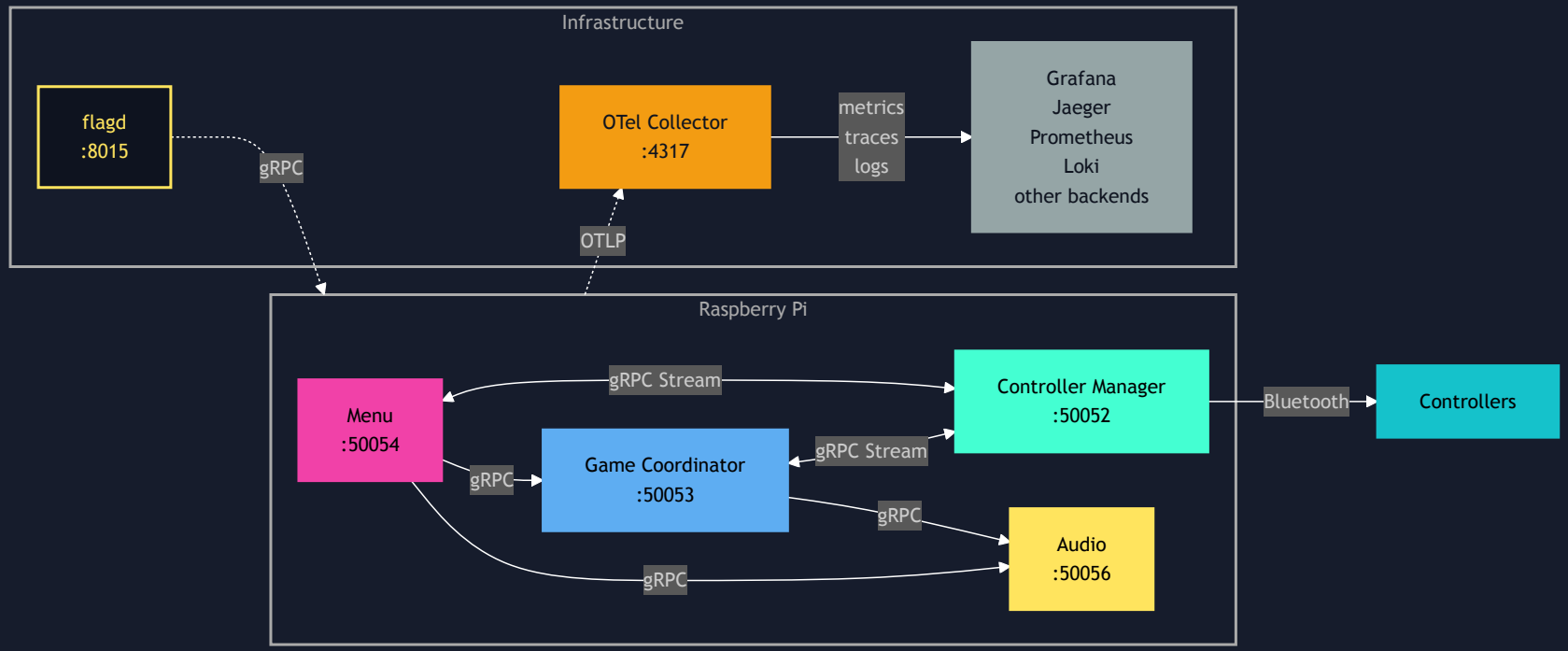
Learning 1: Microservices Unlocked the Stack

Auto-instrumentation came for free with gRPC



W3C trace context propagates automatically, enabling distributed tracing out of the box

Learning 1: Microservices - the full picture

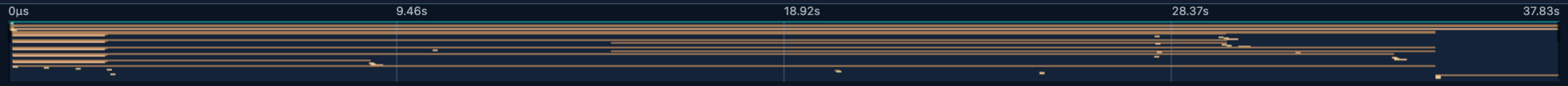


JAEGER UI | Search | Compare | System Architecture | Monitor | Lookup by Trace ID... | About Jaeger

menu-service: start_game 3926df8

Find... | Trace Timeline

Trace Start **March 13 2026, 10:41:58.897** | Duration **37.83s** | Services **4** | Depth **8** | Total Spans **67**

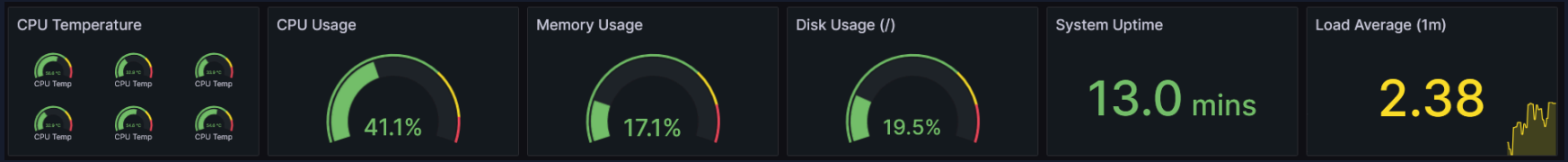


Service & Operation	0µs	9.46s	18.92s	28.37s	37.83s
menu-service start_game					
menu-service stop_lobby_music	1.84ms				
menu-service stop_button_monitor	637µs				
game-coordinator-service StreamGameEvents					
game-coordinator-service Joust Free-for-All					
game-coordinator-service initialization_phase	76.93ms				
game-coordinator-service gameplay_phase	34.7s				
game-coordinator-service players	34.7s				
game-coordinator-service player: Calm Falcon	29.6s				
game-coordinator-service countdown_phase	2.32s				
controller-manager-service effect_countdown	2.25s				
controller-manager-service effect_player_warning	157µs				
controller-manager-service effect_player_warning	146µs				
audio-service PlaySound:Explosion34	52.32ms				
controller-manager-service effect_player_death	301.2ms				
game-coordinator-service player: Swift Tiger	29.62s				
game-coordinator-service countdown_phase	2.32s				
controller-manager-service effect_countdown	2.25s				
! game-coordinator-service controller_poll_de...	15.02s				
controller-manager-service effect_player_warning	132µs				
controller-manager-service effect_player_warning	323µs				

Learning 2: The Raspberry Pi Can Handle It

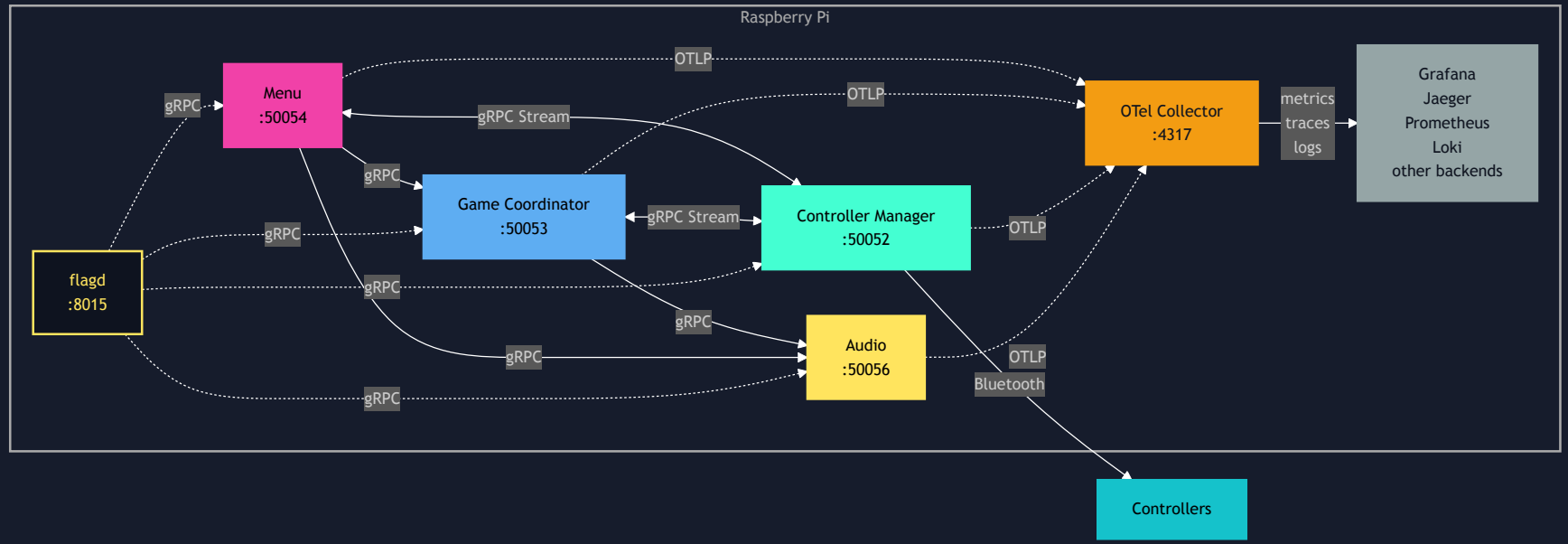
Raspberry Pi 5 Specs

- Quad-core ARM Cortex-A76 @ 2.4GHz
- 8GB LPDDR4X RAM
- ~$80 USD (before global memory shortage)



Learning 2: The Raspberry Pi Can Handle It

The Pi runs both the game AND the full observability stack.



Learning 3: Cardinality Low, Volume High

Export rate is the real challenge

- **Cardinality:** Hundreds of time series, not millions – manageable
- **Volume:** 18 controllers @ 60Hz ≈ 1,080 messages/second

Solution

- **Two levels of batching**
  - SDK buffers and exports on a 100ms schedule (not every event)
  - Collector batches again before forwarding to backends
- **Result:** backends see smooth, regular pushes – not a firehose

Learning 4: Pull Scraping Is Too Slow

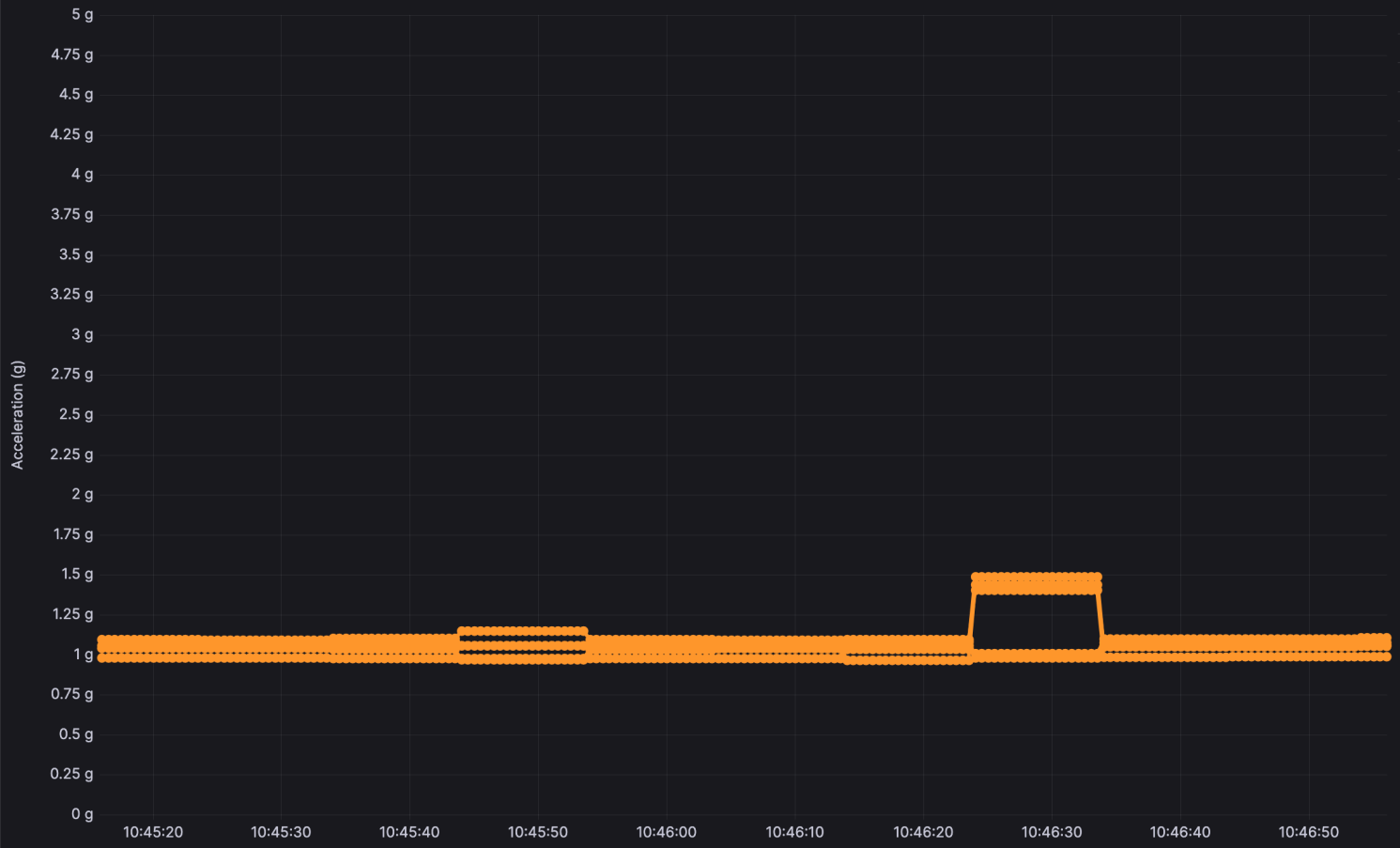
- Controller Manager reads each accelerometer via Bluetooth every **16ms** (60Hz) – miss it → **poll drop**
- Prometheus scrape interval: tuned to **10s**

The gap is enormous

Default Prometheus scrape interval: **60 seconds**
Our game loop: **16 milliseconds**
That's a **3,750× gap** in timing assumptions.

Controller All

Prometheus Pull (10s)



Name	Last *	Max
0006F7D663DA (Pull 10s)	1.11 g	1.15 g
000704241A77 (Pull 10s)	1.06 g	1.44 g
0007048C924A (Pull 10s)	1.06 g	1.49 g
30C3D910F01F (Pull 10s)	0.990 g	1.41 g
E0AE5E58E820 (Pull 10s)	1.08 g	1.08 g

Learning 5: Push Metrics with Prometheus

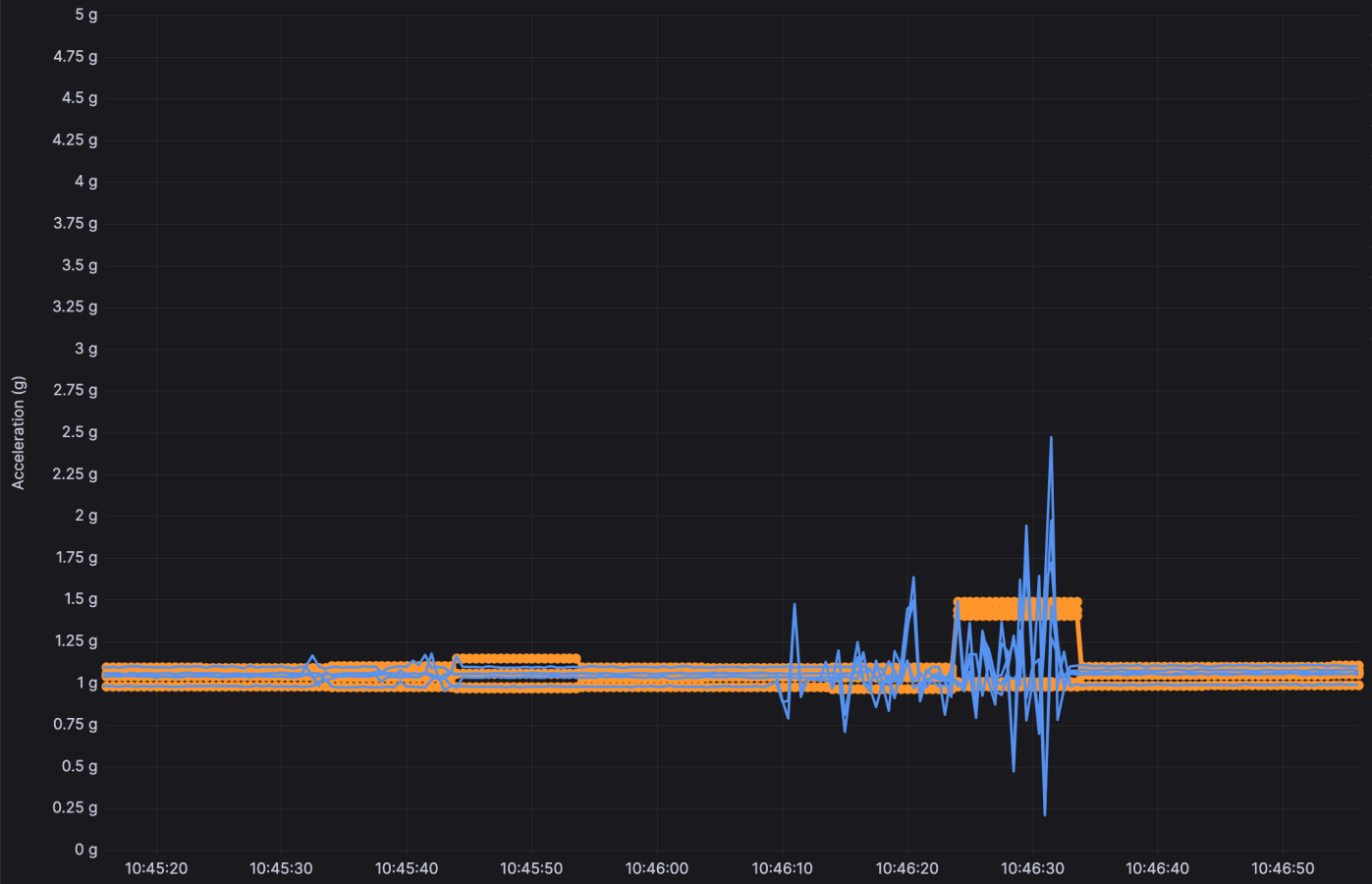
OTLP Push via PeriodicExportingMetricReader

```
# OpenTelemetry SDK Configuration
metric_reader = PeriodicExportingMetricReader(
    exporter=OTLPMetricExporter(endpoint="http://otel-collector:4318"),
    export_interval_millis=flagd.get_int("metrics_export_interval_ms")
    # controller-manager: 100ms (realtime) | other services: 1000ms
)
```

500ms end-to-end reliably – one config change, 30× faster than pull.
For true sub-100ms, the TSDB itself becomes the next bottleneck.

Controller All

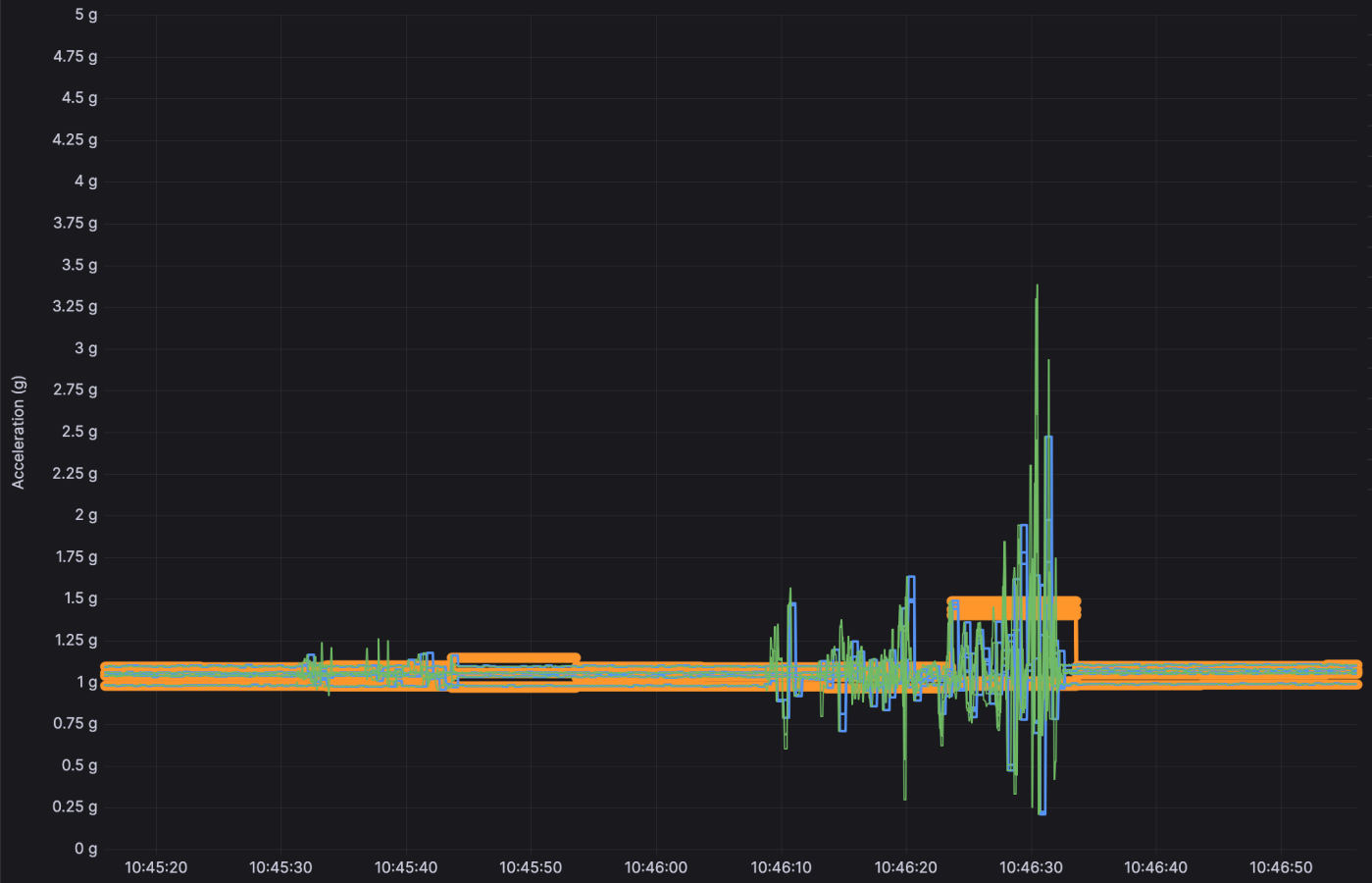
Pull + Push (Prometheus)



Name	Last *	Max
0006F7D663DA (Pull 10s)	1.11 g	1.15 g
000704241A77 (Pull 10s)	1.06 g	1.44 g
0007048C924A (Pull 10s)	1.06 g	1.49 g
30C3D910F01F (Pull 10s)	0.990 g	1.41 g
E0AE5E58E820 (Pull 10s)	1.08 g	1.08 g
0006F7D663DA (OTEL→Prom 500ms)	1.10 g	1.64 g
000704241A77 (OTEL→Prom 500ms)	1.06 g	1.72 g
0007048C924A (OTEL→Prom 500ms)	1.05 g	2.47 g
30C3D910F01F (OTEL→Prom 500ms)	0.990 g	1.97 g
E0AE5E58E820 (OTEL→Prom 500ms)	1.08 g	1.31 g

Controller All

All Three Pipelines



Name	Last *	Max
0006F7D663DA (Pull 10s)	1.11 g	1.15 g
000704241A77 (Pull 10s)	1.06 g	1.44 g
0007048C924A (Pull 10s)	1.06 g	1.49 g
30C3D910F01F (Pull 10s)	0.990 g	1.41 g
E0AE5E58E820 (Pull 10s)	1.08 g	1.08 g
0006F7D663DA (OTEL→Prom 500ms)	1.10 g	1.64 g
000704241A77 (OTEL→Prom 500ms)	1.06 g	1.72 g
0007048C924A (OTEL→Prom 500ms)	1.05 g	2.47 g
30C3D910F01F (OTEL→Prom 500ms)	0.990 g	1.97 g
E0AE5E58E820 (OTEL→Prom 500ms)	1.08 g	1.31 g
0006F7D663DA (OTEL→VM 100ms)	1.11 g	3.30 g
000704241A77 (OTEL→VM 100ms)	1.06 g	2.29 g
0007048C924A (OTEL→VM 100ms)	1.05 g	2.47 g
30C3D910F01F (OTEL→VM 100ms)	0.991 g	2.39 g
E0AE5E58E820 (OTEL→VM 100ms)	1.08 g	3.38 g

Learning 6: Labels Are Not Free

Add a `game_id` label

1,216 series → 18,000 series

Prometheus concurrent p50

87ms → 389ms

VictoriaMetrics: 47ms → 46ms

Every label dimension multiplies your series count – cardinality is where Prometheus hurts

Benchmark: 36 controllers at 100ms push intervals – 2× our setup, so real-world numbers are better

The 6 Learnings

1. **Microservices unlocked auto-instrumentation** – gRPC gave us distributed tracing for free
2. **The Raspberry Pi can run the full stack** – game + entire observability suite on $80 hardware
3. **Cardinality is manageable, volume is the challenge** – two levels of batching tame the firehose
4. **Pull scraping is too slow** – 600 game frames between each Prometheus data point
5. **Push metrics: 500ms end-to-end** – one config change, 30× faster than pull
6. **Labels are not free** – adding `game_id` turned 1,216 series into 18,000 and 4× read latency

Demo: *Real-Time* Observability with CNCF Tools

Watch metrics respond in real-time as we change the game

Controlled Chaos with Feature Flags

Status: none

INJECT FAULT

Poll Drop | Acceleration Spike

LED Flicker | Disconnect

FRACTION: 50%

Reset All Faults

RAW FLAGD STATUS


```
{
  "state": "ENABLED",
  "variants": {
    "accel_spike": "accel_spike",
    "disconnect": "disconnect",
    "led_failure": "led_failure",
    "none": "none",
    "poll_drop": "poll_drop"
  },
  "defaultVariant": "none"
}
```

The Path Forward

What's missing and how you can help

The tools exist. The patterns exist. What's missing is *you*.

These tools work for real-time.

But they're optimized for web apps.

With tuning, you can get subsecond observability on an $80 computer.

What's missing is the documented path.

Real-time systems are everywhere – games, robotics, IoT sensors, industrial controllers. But CNCF tooling docs show web app examples almost exclusively.

Try these tools on real-time systems. Document what works. Contribute examples.

JoustMania is Open Source

Fork it. Break it. Make it better.

Shoutouts

J. S. Joust · JoustMania · OTel / CNCF community

github.com/WatchMeJoustMyFlags/JoustMania

Thank You!

Simon Schrottner

Dynatrace

linkedin.com/in/aepfli

@aepfli · github.com/aepfli

Manuel Timelthaler

Tractive

linkedin.com/in/manuel-timelthaler

github.com/Lorti

or find us on CNCF's Slack Channel

Questions? 🎮

We need your feedback 🙏

sched.co/2CW7W